

Data Generation for Computational Experiments

Purpose

This document describes how the computational test instances in this repository were generated. It is intended to support reproducibility for the exact algorithms and approximation algorithms evaluated in the computational experiments.

The released instance files use the naming convention Instance_ m _ n _ k .txt, where m is the length of the planning horizon, n is the number of orders, and k is the replicate index for a fixed (m, n) combination.

The corresponding C++ data-generation code is provided in data_generation/Generate-data.cpp.

Problem Parameters

Each test instance consists of an order set N with $n = |N|$ orders. Each order i is associated with an order quantity q_i , a committed delivery deadline d_i , and a rejection cost r_i . The remaining problem parameters are the planning horizon length m , production capacity c , the parameters α and β in the shipping cost functions, and the unit inventory holding cost h .

For the common parameters shared with the special case without inventory holding costs, the design follows Li et al. (2022). The generation procedure is supplemented with additional parameters for inventory holding and order rejection costs.

Instance Generation Procedure

For each selected pair (m, n) , generate each instance as follows. Random selections are made independently unless otherwise specified.

1. Order quantities. For each i in $\{1, 2, \dots, n\}$, randomly select q_i from $\{1, 2, \dots, 10\}$.
2. Delivery deadlines. For each i in $\{1, 2, \dots, n\}$, randomly select d_i from $\{1, 2, \dots, m\}$.
3. Production capacity. Compute $c_{min} = \max_{t \in T} \lceil \sum_{i \in N: d_i \leq t} q_i / t \rceil$. Then randomly draw the integer production capacity c from $[0.9 c_{min}, \dots, 1.0 c_{min}]$. This setting allows feasible solutions in which some orders may be rejected.
4. Shipping-cost parameters. Randomly select β from $\{1, 2, \dots, 5\}$. To ensure $\alpha - \beta(m - 1) > 0$, randomly select α from $\{(m - 1)\beta + 1, \dots, 2(m - 1)\beta\}$.
5. Inventory holding cost. Randomly select h from $\{[0.5\beta], [0.5\beta] + 1, \dots, [\beta]\}$.
6. Rejection costs. For each i in $\{1, 2, \dots, n\}$, set $r_i = (\alpha_i + h) q_i$, where α_i is an integer randomly drawn from $[\alpha, 2\alpha]$. This makes the rejection cost increase with the order quantity.

Exact-Algorithm Instances

The exact-algorithm experiments evaluate the exact algorithm and the bounded dynamic programming algorithm under bounded planning horizons. Since the exact algorithm runs in pseudo-polynomial time when m is bounded by a constant, the planning horizon is selected from a small set.

Parameter	Values
Planning horizon m	$\{3, 4, 5\}$
Number of orders n	$\{20, 40, 60, 80, 100\}$
Replicates per (m, n)	10
Total instances	$3 \times 5 \times 10 = 150$
Repository directory	instances/Instances-for-Exact-Algorithm

All other parameters are generated according to the procedure above.

Approximation-Algorithm Instances

The approximation-algorithm experiments use a wider set of planning horizons while keeping the same order-size grid and the same number of replicates per parameter combination.

Parameter	Values
Planning horizon m	{5, 10, 15}
Number of orders n	{20, 40, 60, 80, 100}
Replicates per (m, n)	10
Total instances	$3 \times 5 \times 10 = 150$
Repository directory	instances/Instances-for-Approximation-Algorithm

All other parameters are generated according to the procedure above.